

Competitive Programming

Section 58: Full Virtual Contest Workflow

Competitive Programming Course

2026-07-06

Table of contents

1	Contest execution	3
1.1	First 10 minutes	3
1.2	During the contest	3
1.3	Debugging protocol	3
1.4	Penalty management	4
1.5	Post-contest upsolve	4
1.6	Practical triage example	4
1.7	Wrong-answer recovery	4
1.8	Upsolve record template	5
1.9	Time-boxing rules	5
1.10	Contest postmortem categories	5
2	Exercises	6

1 Contest execution

A virtual contest is not just solving. Triage, choose low-risk first solves, manage penalties, and leave time for debugging. The local tasks model ranking, partitioning workload, and quick algorithm selection.

1.1 First 10 minutes

Do a shallow scan of all problems:

- identify implementation-only tasks;
- mark topics you recognize;
- estimate risk, not just difficulty;
- avoid deep reading of a single hard statement too early.

The goal is to build a solve order. A slightly easier problem with high bug risk may be worse than a medium problem with a clean known template.

1.2 During the contest

For each problem, write a small contract before coding:

```
Model:  
Invariant:  
Complexity:  
Corner cases:
```

This takes less than a minute and prevents many wrong submissions.

1.3 Debugging protocol

When a solution fails:

1. Re-read input/output format.
2. Test the smallest nontrivial case.
3. Test boundaries: empty, one item, all equal, sorted, reversed.
4. Compare against brute force if possible.
5. Only then reconsider the algorithm.

Do not rewrite a solution until you know whether the bug is modeling, implementation, or format.

1.4 Penalty management

The best contest strategy is not “submit fast”; it is “submit when the proof and stress tests agree.” For medium problems, one extra local test is often cheaper than a wrong-answer penalty plus context switch.

1.5 Post-contest upsolve

For every unsolved problem:

1. Solve without editorial for a fixed time box.
2. Read the smallest useful hint.
3. Implement from your own understanding.
4. Write down the missed pattern.
5. Add one template or checklist item if it would have helped.

This converts contest pain into future speed.

1.6 Practical triage example

After a scan, mark problems like this:

```
A: implementation, 10 min, low risk
B: binary search, 25 min, medium risk
C: geometry, 35 min, high bug risk
D: unknown, skip for now
E: known DP but long, save after A/B
```

A reasonable order is:

```
A -> B -> E or C depending on confidence -> D last
```

The key is not to solve strictly by statement order. Solve by expected accepted submissions per unit time, while preserving enough time to debug.

1.7 Wrong-answer recovery

When a submission fails, write one sentence:

```
I believe the bug is in model / edge case / implementation / output format.
```

Then test for that category. Randomly changing code after WA is usually slower than classifying the failure.

1.8 Upsolve record template

After the contest, record:

```
Problem:
Missed because:
Key observation:
Algorithm:
Proof idea:
Bug risk:
Reusable pattern:
```

The goal is to avoid solving the same problem from zero next month. A red-coder training log is a pattern database, not a diary.

1.9 Time-boxing rules

A practical solo contest rule:

```
If no model after 10 minutes: park it.
If model exists but proof is shaky after 15 more minutes: park it.
If code is failing for 20 minutes: make a tiny brute force or ask what category
the failure belongs to.
```

The exact numbers can change, but the point is to avoid invisible time loss.

1.10 Contest postmortem categories

Classify every missed problem:

```
knowledge gap
recognition gap
proof gap
implementation bug
testing gap
time-management mistake
```

Each category has a different fix. Reading more editorials does not fix an implementation bug; writing more templates does not fix a recognition gap.

2 Exercises

- A. Scoreboard Rank: Sort teams by more solved, then lower penalty, then name.
- B. Split Workload: Split problem times between two solvers minimizing the maximum load.
- C. Quick MST Decision: Use MST savings as a contest modeling drill. # Contest checklist

State the invariant, choose the model, prove the operation, then code the smallest reliable version.